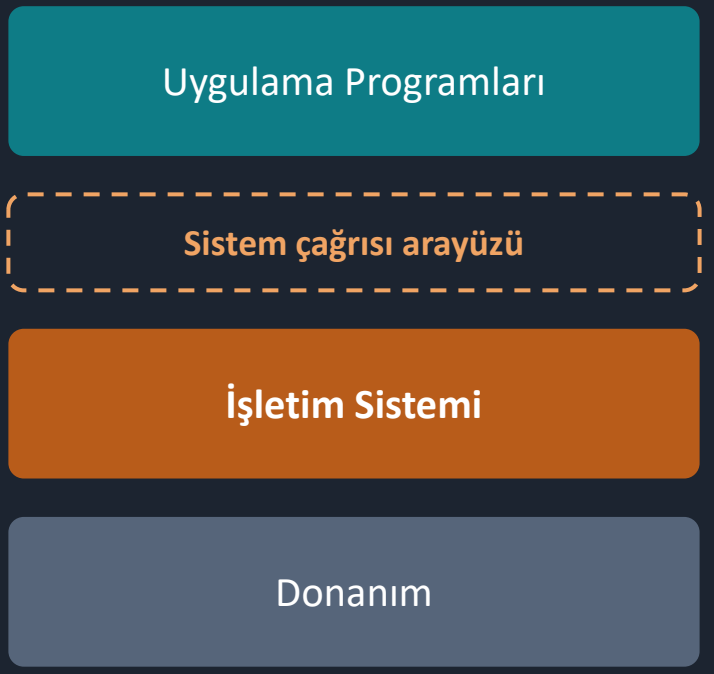
KARADENİZ TEKNİK ÜNİVERSİTESİ

Yazılım Geliştirme Bölümü

İŞLETİM SİSTEMLERİ

Hafta 2

İşletim Sistemi Hizmetleri, Sistem Çağrıları ve İşletim Sistemi Yapıları



Dr. Öğr. Üyesi Sayyad Alizadeh

Bu Haftanın Yol Haritası

1

İS hizmetleri

İşletim sistemi programlara ve kullanıcılara ne sunar?

2

Kullanıcı arayüzleri

Kullanıcı bu hizmetlere nasıl ulaşır?

3

Sistem çağrıları

Bir program çekirdekten nasıl hizmet ister?

4

Sistem çağrısı türleri

Hangi tür hizmetler istenebilir?

5

Sistem programları

Kullanıcının gördüğü İS ortamı nedir?

6

Tasarım ve gerçekleştirme

Politika ile mekanizma neden ayrılır?

7

İS yapıları

Monolitik, katmanlı, mikroçekirdek, modüler, hibrit

8

Önyükleme

Sistem açılışta çekirdeği nasıl bulur ve başlatır?

Öğrenme Çıktıları

Bu haftanın sonunda öğrenciler:

1. İşletim sistemi hizmetlerini kullanıcıya ve sisteme yönelik olarak sınıflandırır
2. Kabuk ile çekirdek arasındaki farkı açıklar
3. Bir sistem çağrısının kullanıcı modundan çekirdek moduna ve geri yolculuğunu adım adım anlatır
4. API ile sistem çağrısını ayırt eder; parametre aktarma yöntemlerini karşılaştırır
5. Sistem çağrısı türlerini ve sistem programlarını örneklerle açıklar
6. Politika–mekanizma ayrımını ve beş temel İS yapısını karşılaştırır

Geçen Haftadan Bu Haftaya

HAFTA 1'DE ÖĞRENDİKLERİMİZ

- **Kullanıcı modu / Çekirdek modu**
- **Ayrıcalıklı komutlar**
- **Kesme (Interrupt) ve Tuzak (Trap)**
- **Donanım Zamanlayıcısı (Timer)**

- Kullanıcı programları ayrıcalıklı komutları yürütemez
- G/Ç aygıtlarına ve donanıma doğrudan erişemez
- Ama dosya açmak, ekrana yazmak, yeni program başlatmak ister

Hafta 1 konuları yeniden anlatılmaz; yalnızca bu haftanın temeli olarak hatırlatılır.

BU HAFTANIN TEMEL SORUSU

Bir kullanıcı programı çekirdek modunda çalışamıyorsa, işletim sisteminden nasıl hizmet ister?

Yanıt: Sistem Çağrısı (System Call) — denetli bir kapıdan çekirdeğe geçiş

İşletim Sistemi Hizmetlerine Genel Bakış



Silberschatz vd., Şekil 2.1 temel alınarak yeniden çizilmiştir

Kullanıcıya Yardımcı Olan Hizmetler

1 Kullanıcı arayüzü

Komut satırı, grafik ya da dokunmatik etkileşim

Senaryo: Terminalde bir komut yazmak

2 Program yürütme

Programı belleğe yükleme, çalıştırma, normal ya da hatalı sonlandırma

Senaryo: Bir IDE'de “Çalıştır” düğmesine basmak

3 G/Ç işlemleri

Kullanıcılar aygıtları doğrudan denetlemez; G/Ç'yi İS yapar

Senaryo: Bir programın ekrana çıktı vermesi

4 Dosya sistemi işlemleri

Dosya ve dizin oluşturma, silme, okuma, yazma, arama; izin yönetimi

Senaryo: Bir programın yapılandırma dosyasını açması

5 İletişim

Süreçler arası bilgi alışverişi: paylaşımlı bellek ya da mesaj geçirme

Senaryo: Tarayıcının bir sunucuyla veri alışverişi

6 Hata tespiti

CPU, bellek, G/Ç ve programlardaki hataları sürekli izleme

Senaryo: Geçersiz bellek erişiminde programın sonlandırılması

Kaynak: Silberschatz vd., Bölüm 2.1

Sistemin Verimli Çalışmasını Sağlayan Hizmetler

Kaynak ayırma	Birden çok kullanıcı ya da iş aynı anda çalışırken CPU zamanı, bellek, depolama ve aygıtlar paylaştırılır	**Senaryo:** Aynı anda çalışan iki program da CPU ve bellek ister: kime, ne kadar?
Hesap tutma	Hangi kullanıcının hangi kaynaktan ne kadar kullandığı kaydedilir: faturalama ya da istatistik için	**Senaryo:** Bir sunucuda kullanıcı başına CPU kullanımının raporlanması
Koruma ve güvenlik	Kaynaklara tüm erişimler denetlenir; sisteme girişte kimlik doğrulama istenir	**Senaryo:** Bir sürecin başka bir sürecin belleğine erişmesinin engellenmesi

Bu hizmetler kullanıcıya doğrudan bir işlev sunmaz; çok kullanıcılı ve çok programlı sistemin adil, verimli ve güvenli çalışmasını sağlar.

Kullanıcı İşletim Sistemiyle Nasıl Etkileşir?

Komut satırı (CLI)

Komut yorumlayıcısı (Command Interpreter) / kabuk (Shell)

- Metin komutlarıyla çalışır
- Tekrarlanan işler betiklerle (shell script) otomatikleştirilebilir
- Sistem yöneticileri ve ileri kullanıcılar tarafından sık tercih edilir

UNIX/Linux: bash, sh, csh…

Grafik arayüz (GUI)

Masaüstü benzetmesi: pencere, simge, menü

- Fare ve klavye ile etkileşim
- Öğrenmesi daha kolaydır
- Bazı işlevler yalnızca komut satırından erişilebilir olabilir

Windows, macOS (Aqua), GNOME, KDE

Dokunmatik arayüz

Mobil cihazlar için

- Hareketlerle (dokunma, kaydırma) etkileşim
- Fare mobil cihazlarda pratik değildir
- Klavye çoğunlukla ekranda benzetilir

iOS, Android

Dördüncü biçim: toplu işlem arayüzü — komutlar bir dosyaya yazılır ve dosya yürütülür. Kaynak: Silberschatz vd., Bölüm 2.2

Komut Yorumlayıcısı Bir Komutu Nasıl Yürütür?

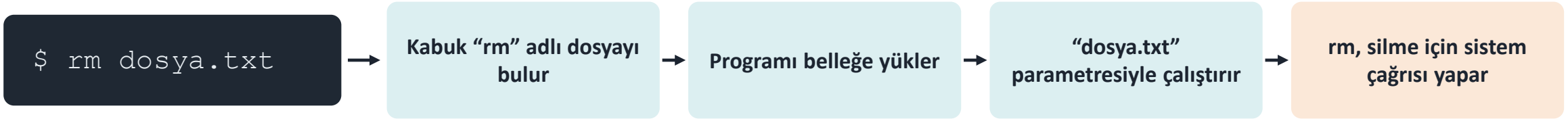
Ana görevi: kullanıcının yazdığı bir sonraki komutu almak ve yürütmek. İki genel yaklaşım vardır:

Yaklaşım 1: Kod kabuğun içinde

- Her komutun kodu komut yorumlayıcısının kendi içindedir
- Komut sayısı arttıkça yorumlayıcı büyür

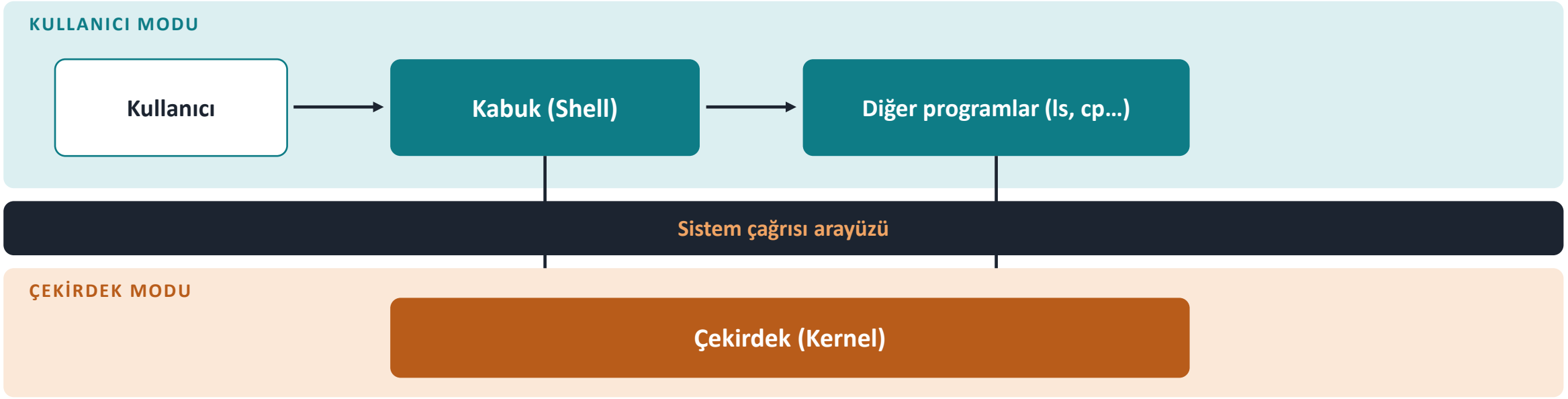
Yaklaşım 2: UNIX'te çoğu komut = ayrı bir sistem programı

- Kabuk komutu “anlamaz”; adını, yüklenecek dosyayı bulmak için kullanır
- Yeni komut eklemek için yeni bir dosya yeterlidir; kabuk değişmez
- Not: cd gibi bazı komutlar kabuğun kendi içinde yerleşik (shell built-in) olabilir.



Kaynak: Silberschatz vd., Bölüm 2.2.1 · Kabuğun süreç oluşturma ayrıntısı (fork, exec) Hafta 3'te

Kabuk ile Çekirdek Aynı Şey Değildir



Kabuk

Kullanıcı ile İS arasında komut arayüzü sunan sıradan bir programdır; kullanıcı modunda çalışır ve değiştirilebilir.

Çekirdek

İşletim sisteminin her zaman çalışan, çekirdek modundaki kısmıdır; kabuk dahil tüm programlara hizmet verir.

Tanenbaum & Bos (1.5.6): kabuk, İS özelliklerini yoğun kullanır ama işletim sisteminin parçası değildir

Sistem Çağrısı (System Call) Nedir?

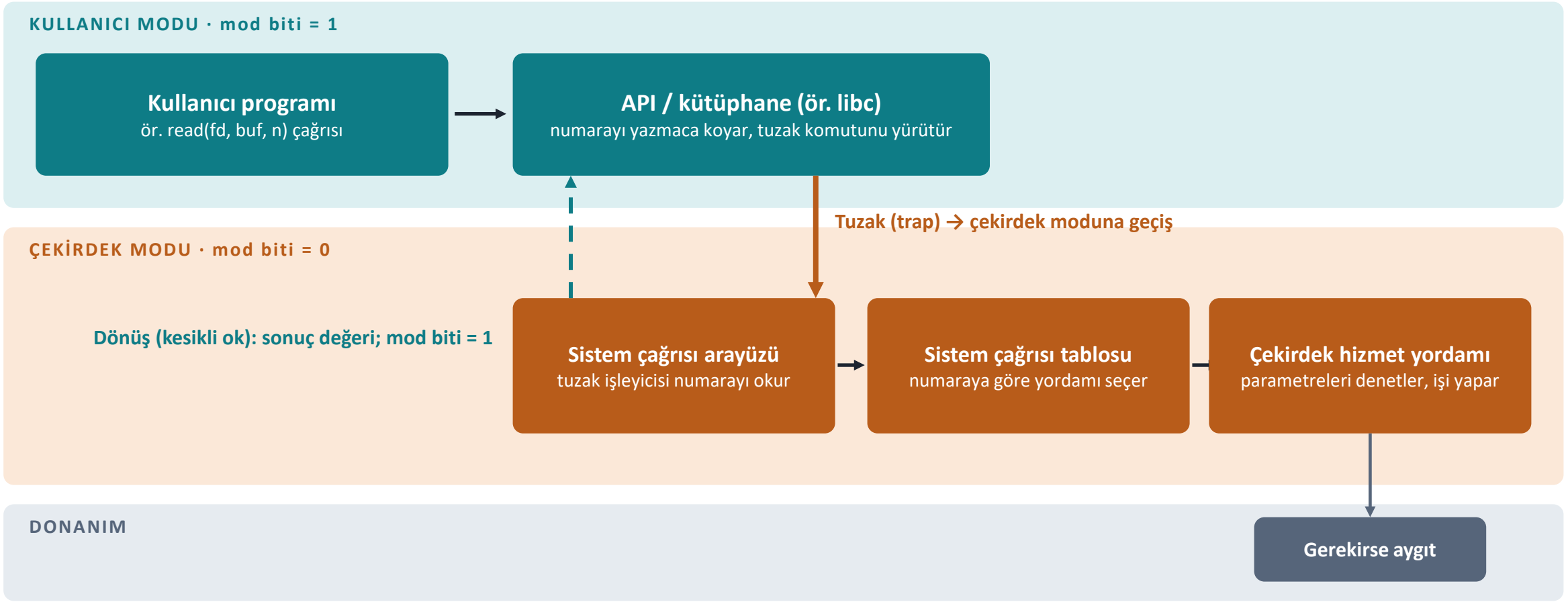
Sistem çağrısı: Çalışan bir programın işletim sisteminden hizmet istemesini sağlayan, kullanıcı modundan çekirdek moduna denetimli geçiş mekanizmasıdır.

Sıradan bir işlev çağrısından farkı

Ölçüt	Sıradan işlev çağrısı	Sistem çağrısı
Mod değişir mi?	Hayır, kullanıcı modunda kalınır	Evet: kullanıcı modu → çekirdek modu
Hedef adres	Program istediği adrese atlayabilir	Yalnızca çekirdeğin belirlediği sabit giriş noktası
Kullanılan komut	Sıradan çağrı (call) komutu	Tuzak (trap) ya da özel sistem çağrısı komutu
Parametreler	Doğrudan kullanılır	Çekirdek, geçerliliğini denetledikten sonra kullanır

Kaynaklar: Silberschatz vd., Bölüm 1.5.1 ve 2.3; Tanenbaum & Bos, Bölüm 1.6

Bir Sistem Çağrısının Yolculuğu



Kavramsal şema · Kaynaklar: Silberschatz vd., Şekil 2.6; Tanenbaum & Bos, Şekil 1-17

Adım Adım: read() Çağrısı Sırasında Ne Olur?

```
count = read(fd, buffer, nbytes);
```

1. Program, parametreleri hazırlar ve kütüphanedeki read() işlevini çağırır
2. Kütüphane, read için sistem çağrısı numarasını bir yazmaca koyar
3. Tuzak komutu yürütülür: donanım çekirdek moduna geçer, sabit giriş noktasına atlar
4. Çekirdek numarayı inceler ve tablo üzerinden ilgili işleyiciye dağıtır
5. İşleyici parametreleri denetler ve okuma işini yapar
6. Kullanıcı moduna dönülür; kütüphane sonucu programa döndürür

● kullanıcı modu ● çekirdek modu

Dönüş değeri (count)

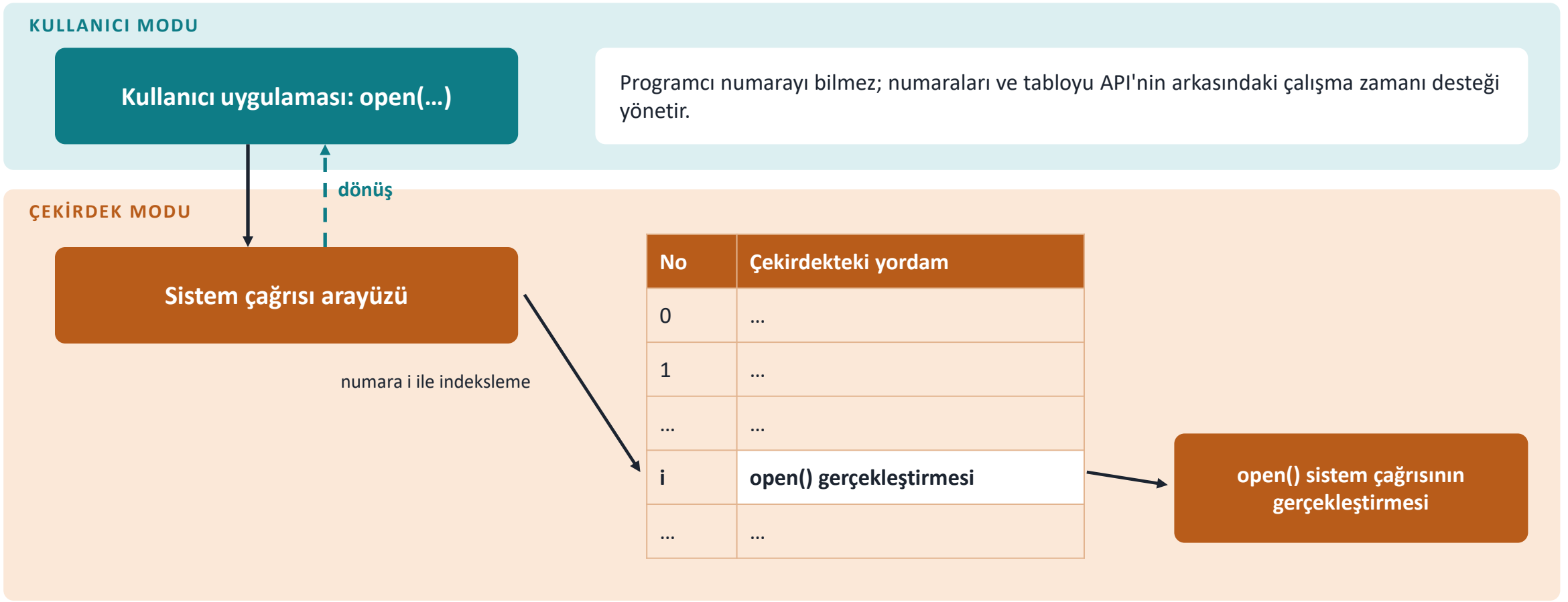
- > 0 : gerçekten okunan bayt sayısı
- 0 : dosya sonu
- −1 : hata; hata kodu errno değişkenine yazılır

Neden bu kadar adım?

Çekirdek, kullanıcı programına güvenmez: giriş noktası sabittir ve parametreler denetlenir. Program her zaman sonucu denetlemelidir.

Tanenbaum & Bos, Bölüm 1.6'daki 11 adımlı read örneği sadeleştirilmiştir

Sistem Çağrısı Numarası ve Sistem Çağrısı Arayüzü



Silberschatz vd., Şekil 2.6 temel alınarak yeniden çizilmiştir

Uygulama Programlama Arayüzü (API)

API (Application Programming Interface): Programcıya sunulan işlevler kümesi; her işlevin parametreleri ve dönüş değerleri tanımlıdır. Programcı API'ye, İS'nin sağladığı bir kod kütüphanesi üzerinden erişir.

Yaygın üç API

Windows API	Windows sistemleri
POSIX API	UNIX, Linux, macOS gibi POSIX tabanlı sistemler · C için kütüphane: libc
Java API	Java sanal makinesinde çalışan programlar

Neden doğrudan sistem çağrısı yerine API?

Taşınabilirlik

Aynı API'yi destekleyen sistemlerde program derlenip çalışabilir.

Kullanım kolaylığı

Gerçek sistem çağrıları çoğu zaman daha ayrıntılı ve kullanımı daha zordur.

Kaynak: Silberschatz vd., Bölüm 2.3

Her API Çağrısı Doğrudan Bir Sistem Çağrısı Değildir

API

- Programcının gördüğü işlev arayüzü
- Kullanıcı modunda, bir kütüphanede çalışır
- Bir sözleşmedir: ne verilir, ne döner?

Sistem çağrısı

- Çekirdeğin sunduğu hizmete geçiş mekanizması
- Tuzak ile çekirdek moduna geçer
- Her İS'nin kendi çağrıları ve adları vardır

API işlevi ile sistem çağrısı arasındaki ilişki değişebilir:

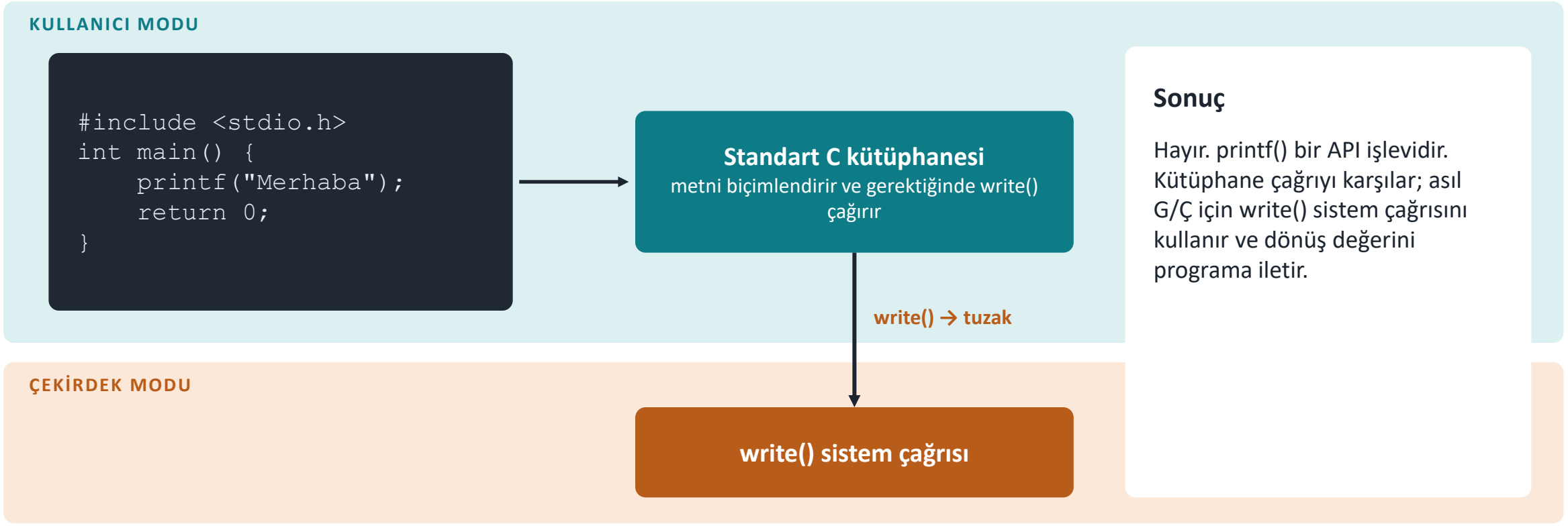
Yaklaşık bire bir
UNIX'te read() kütüphane işlevi, read sistem çağrısını çağırır

Farklı adla, arka planda
Windows'ta CreateProcess(), çekirdekteki NTCreateProcess() sistem çağrısını çağırır

Sistem çağrısız
Bazı kütüphane işlevleri (ör. strlen) hiç çekirdeğe girmez

Silberschatz vd., Bölüm 2.3 · Tanenbaum & Bos, Bölüm 1.6.5: Win32 API ile gerçek sistem çağrıları büyük ölçüde birbirinden ayrılmıştır

Örnek: printf() Doğrudan Çekirdeğe Girer mi?



Silberschatz vd., Bölüm 2.4 (Standart C kütüphanesi örneği) temel alınarak yeniden çizilmiştir

Örnek: Bir Dosyayı Okuyup Yazmak

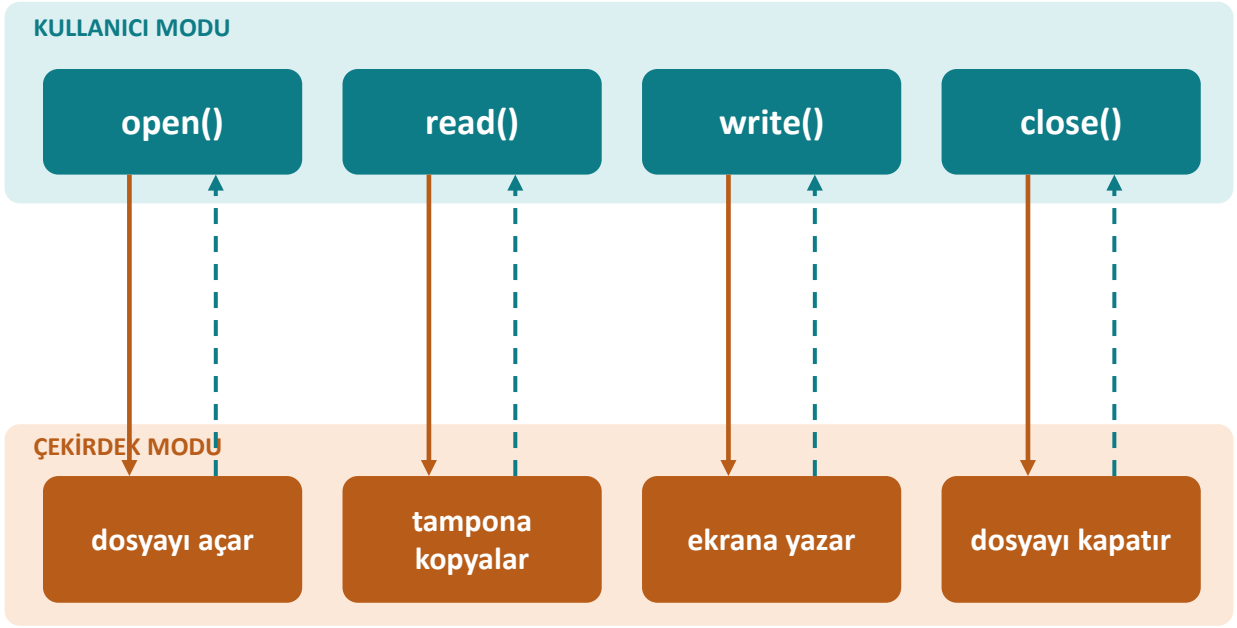
```
fd = open("veri.txt", O_RDONLY);
n  = read(fd, buf, 512);
write(1, buf, n);
close(fd);
```

Kısa kod; dört kez çekirdeğe geçiş (fd 1 = standart çıktı, ekran)

read() API tanımı (POSIX)

ssize_t read(int fd, void *buf, size_t count)

fd: okunacak dosya tanımlayıcısı · buf: verinin yazılacağı tampon · count: en fazla okunacak bayt



Her çağrıda: kullanıcı modu → çekirdek modu → kullanıcı modu

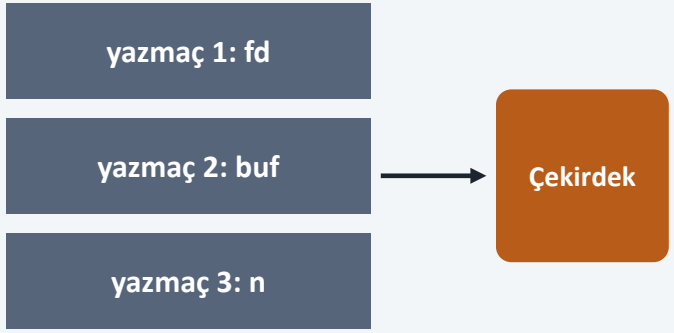
Turuncu ok: tuzak ile çekirdeğe geçiş · Kesikli ok: sonuçla dönüş

Kaynaklar: Silberschatz vd., Bölüm 2.3 (read API örneği); Tanenbaum & Bos, Bölüm 1.6.2

Sistem Çağrısına Parametre Aktarma: Üç Yöntem

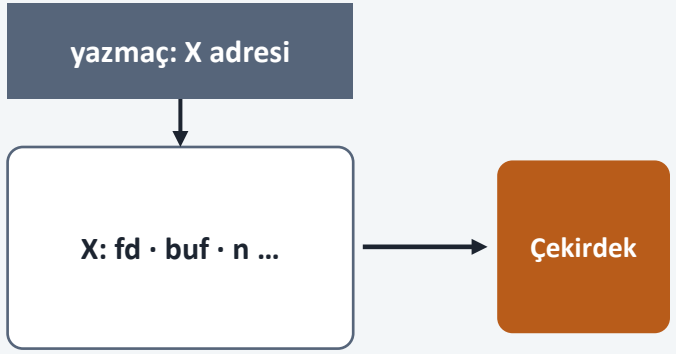
Çekirdek, istenen çağrının numarasından fazlasını bilmelidir: hangi dosya, hangi tampon adresi, kaç bayt?

1 Yazmaçlar



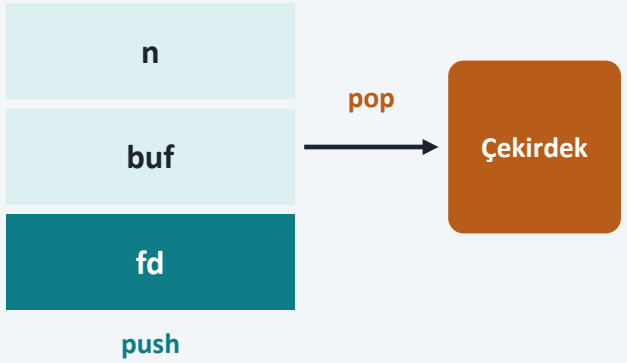
En basit ve en hızlı yöntem. Sınırı: parametre sayısı yazmaç sayısını aşabilir.

2 Bellekte blok / tablo



Parametreler bellekteki bir blokta tutulur; yazmaçla yalnızca bloğun adresi aktarılır. Linux ve Solaris bu yaklaşımı kullanır.

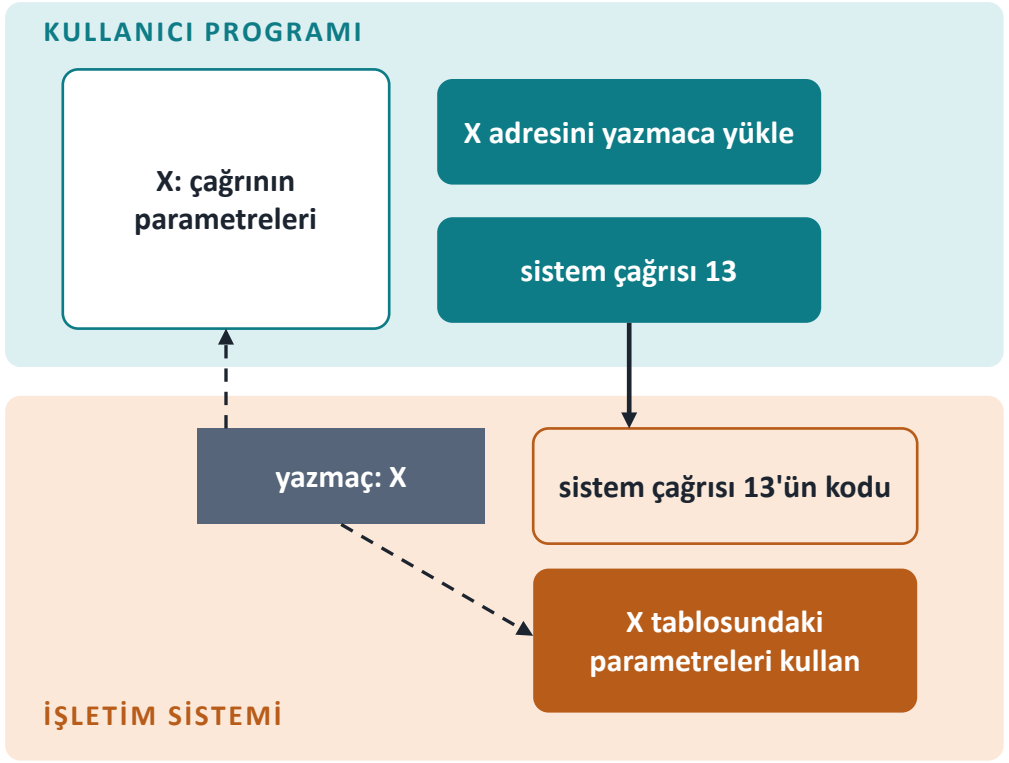
3 Yığın (Stack)



Program parametreleri yığına iter (push); işletim sistemi yığından alır (pop). Blok gibi sayı ve uzunluk sınırı getirmez.

Kaynak: Silberschatz vd., Bölüm 2.3

Bellekte Tablo Yöntemi ve Yöntemlerin Karşılaştırması



Silberschatz vd., Şekil 2.7 temel alınmıştır

Yöntem	Güçlü yanı	Sınırlaması
Yazmaçlar	En hızlı; bellek erişimi gerektirmez	Yazmaç sayısı kadar parametre
Bellekte blok	Parametre sayısı/uzunluğu sınırlı değil	Ek bellek erişimi
Yığın	Parametre sayısı/uzunluğu sınırlı değil	Ek bellek erişimi

Bazı işletim sistemleri, parametre sayısını ve uzunluğunu sınırlamadığı için blok ya da yığın yöntemini tercih eder.

Sistem Çağrılarının Altı Temel Türü

Süreç denetimi

sonlandır, iptal et · yükle, yürüt · süreç oluştur / sonlandır · olay bekle / bildir

Aygıt yönetimi

aygıt iste / bırak · oku, yaz, konumlan · aygıt özniteliği al / ayarla

İletişim

bağlantı oluştur / sil · mesaj gönder / al · paylaşımlı bellek oluştur



Dosya yönetimi

oluştur, sil · aç, kapat · oku, yaz, konumlan · öznitelik al / ayarla

Bilgi bakımı

saat / tarih al, ayarla · sistem verisi al · süreç bilgisi al / ayarla

Koruma

izin al / ayarla · kullanıcıya erişim ver / reddet

Genel (soyut) çağrı adlarıdır; her İS'nin kendi adları vardır. Kaynak: Silberschatz vd., Bölüm 2.4 ve Şekil 2.8

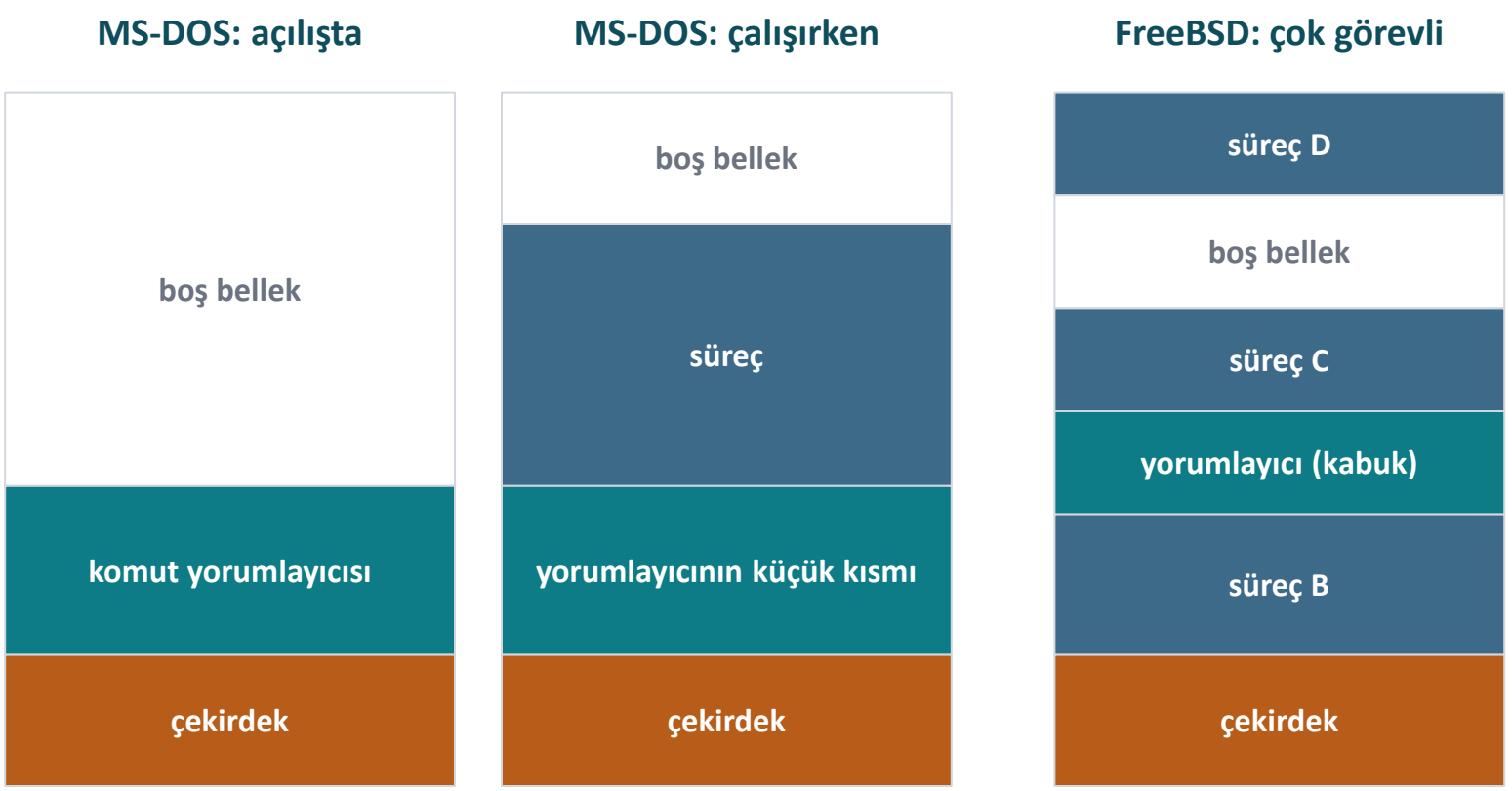
Gerçek Sistemlerde Karşılıkları: Windows ve UNIX

Tür	Windows	UNIX
Süreç denetimi	`CreateProcess()` · `ExitProcess()`	`fork()` · `exit()` · `wait()`
Dosya yönetimi	`CreateFile()` · `ReadFile()` · `CloseHandle()`	`open()` · `read()` · `close()`
Aygıt yönetimi	`SetConsoleMode()` · `ReadConsole()`	`ioctl()` · `read()` · `write()`
Bilgi bakımı	`GetCurrentProcessID()` · `Sleep()`	`getpid()` · `alarm()` · `sleep()`
İletişim	`CreatePipe()` · `MapViewOfFile()`	`pipe()` · `shm_open()` · `mmap()`
Koruma	`SetFileSecurity()`	`chmod()` · `umask()` · `chown()`

UNIX'te dosyalar ve aygıtlar için büyük ölçüde aynı çağrılar (read, write) kullanılır: aygıtlar özel dosyalar gibi görünür.

Seçilmiş örnekler. Kaynak: Silberschatz vd., Bölüm 2.4 (Windows ve UNIX sistem çağrısı örnekleri)

Süreç Denetimi Örneği: Tek Görevli ve Çok Görevli Sistem



- MS-DOS yeni süreç oluşturmaz: programı yükler, yorumlayıcının çoğunun üzerine yazar
- FreeBSD'de kabuk çalışmaya devam eder; yeni süreç fork() ile oluşturulur, program exec() ile yüklenir
- Süreç bitince exit() ile durum kodunu (0 ya da hata kodu) döndürür

Silberschatz vd., Şekil 2.9–2.10 temel alınmıştır · fork() ve exec() ayrıntısı Hafta 3'te

Dosya Kopyalama: Basit Bir İşin Arkasındaki Sistem Çağrıları

1	Kaynak dosya adını al	→	`ekrana yaz + klavyeden oku`
2	Hedef dosya adını al	→	`ekrana yaz + klavyeden oku`
3	Kaynak dosyayı aç	→	`open()`
4	Hedef dosyayı oluştur	→	`create()`
5	Döngü: oku → yaz	→	`read()` · `write()`
6	Dosyaları kapat	→	`close()`
7	Tamamlandı mesajı, normal sonlan	→	`write()` · `exit()`

Her adımın hatası da sistem çağrısı gerektirir

Kaynak dosya yok ya da korumalı
→ mesaj yaz, anormal sonlan

Hedef dosya zaten var
→ iptal et ya da sil ve yeniden oluştur

Okuma hatası / dosya sonu
→ döngüden çık

Diskte yer kalmadı
→ hata bildir

Programcı bunları API ile görür; sistemler saniyede binlerce sistem çağrısı yürütür.

Silberschatz vd., Şekil 2.5 temel alınmıştır (çağrı adları genel örnektir)

Sistem Programları (System Programs)



Sistem programı (sistem yardımcı programı / System Utility): Program geliştirme ve yürütme için uygun bir ortam sağlayan, İS ile birlikte gelen programlardır.

- Bazıları sistem çağrılarına basit bir kullanıcı arayüzüdür; bazıları çok daha karmaşıktır
- Kullanıcı modunda çalışan sıradan programlardır
- Çoğu kullanıcının gördüğü İS, sistem çağrıları değil, uygulama ve sistem programlarıdır

Kaynak: Silberschatz vd., Bölüm 2.5

Sistem Programı Kategorileri

Dosya yönetimi

Dosya ve dizin oluşturma, silme, kopyalama, listeleme

`ls · cp · rm`

Durum bilgisi

Tarih, saat, boş bellek ve disk alanı, performans ve günlük bilgisi

`date · top · df`

Dosya değiştirme

Metin düzenleyiciler; dosya içeriğinde arama ve dönüştürme

`nano · vim · grep`

Programlama dili desteği

Derleyiciler, çeviriciler, hata ayıklayıcılar, yorumlayıcılar

`gcc · gdb · python`

Program yükleme ve yürütme

Yükleyiciler, bağlama düzenleyicileri, hata ayıklama sistemleri

`ld · ld.so`

İletişim

Süreçler, kullanıcılar ve bilgisayarlar arasında bağlantı: uzaktan oturum, dosya aktarımı

`ssh · scp`

Arka plan hizmetleri

Açılışta başlayan, sürekli çalışan hizmetler (servis, alt sistem, daemon)

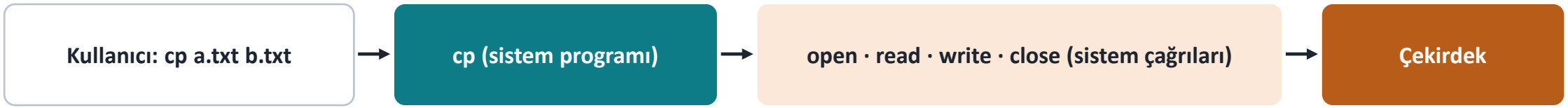
`sshd · cupsd`

Örnek komutlar Linux/UNIX ortamından seçilmiştir; kategoriler Silberschatz vd., Bölüm 2.5'e göredir.

Sistem Çağrısı ile Sistem Programı Aynı Şey Değildir

Ölçüt	Sistem çağrısı	Sistem programı
Nedir?	Çekirdeğe giriş arayüzü / mekanizması	İS ortamını sağlayan çalıştırılabilir program
Nerede çalışır?	Hizmet çekirdek modunda yürütülür	Kullanıcı modunda, sıradan süreç olarak
Kim kullanır?	Programlar (çoğunlukla API üzerinden)	Kullanıcılar ve diğer programlar
Örnek	open() · read() · write()	cp · ls · derleyici · kabuk

İLİŞKİ: cp KOMUTU



Sistem programları, işlerini yapmak için sistem çağrılarını kullanır.

Tasarım Hedefleri: Kimin İsteği?

İlk adım: donanım seçimi ve sistem türü (toplu işlem, zaman paylaşımlı, tek/çok kullanıcılı, dağıtık, gerçek zamanlı, genel amaçlı)

Kullanıcı hedefleri

- Kullanışlı, öğrenmesi ve kullanması kolay
- Güvenilir, güvenli ve hızlı

Sistem hedefleri

- Tasarlaması, gerçekleştirmesi ve bakımı kolay
- Esnek, güvenilir, hatasız ve verimli

Bu hedefler belirsizdir ve farklı yorumlanabilir; tek bir doğru çözüm yoktur. Örneğin gömülü gerçek zamanlı VxWorks ile IBM anabilgisayarlarındaki MVS'nin gereksinimleri çok farklıdır.

Kaynak: Silberschatz vd., Bölüm 2.6.1

Politika ve Mekanizma (Policy and Mechanism)

Politika: NE yapılacak?
karar, kural, tercih

↓

Mekanizma: NASIL yapılacak?
kararı uygulayan araç

ÖRNEKLER

Donanım Zamanlayıcısı	**Politika:** Bir kullanıcıya ne kadar süre verilecek?	**Mekanizma:** Süre dolunca kesme üreten zamanlayıcı
CPU zamanlama	**Politika:** Hangi süreç önce seçilecek? Öncelikler nasıl atanacak?	**Mekanizma:** Dispatcher, context switch ve CPU denetimini seçilen sürece aktaran donanım/yazılım mekanizmaları.

Neden ayrılır? Esneklik

Politikalar yere ve zamana göre değişir. Genel bir mekanizma, değişmeden zıt politikaları da destekleyebilir: aynı öncelik mekanizması G/Ç yoğun programları da, CPU yoğun programları da öne alabilir.

Kaynaklar: Silberschatz vd., Bölüm 2.6.2; Tanenbaum & Bos, Bölüm 1.7.3 · CPU zamanlama algoritmaları Hafta 5'te

Gerçekleştirme: İşletim Sistemi Hangi Dille Yazılır?

İlk sistemler
Assembly dili

→

Bugün çoğunlukla
C / C++

→

Sistem programları
C, C++, betik dilleri (Perl, Python, kabuk)

Yüksek düzeyli dilin artıları

- Kod daha hızlı yazılır, daha kısa ve anlaşılırdır
- Hata ayıklaması kolaydır; derleyici gelişimi tüm sisteme yansır
- Başka donanıma taşımak çok daha kolaydır

Olası eksiler

- Hız kaybı ve daha fazla bellek kullanımı
- Bugün büyük bir sorun sayılmaz; darboğaz yordamlar sonradan assembly ile yazılabilir

Örnek: Intel 8088 assembly'siyle yazılan MS-DOS yalnızca x86'da doğal çalışır; büyük ölçüde C ile yazılan Linux pek çok farklı işlemcide çalışır.

Kaynak: Silberschatz vd., Bölüm 2.6.3

İşletim Sistemi Nasıl Yapılandırılır?

Büyük ve karmaşık bir sistem, iyi tanımlanmış girdileri, çıktıları ve işlevleri olan bileşenlere ayrılmalıdır. Soru: bu bileşenler çekirdekte nasıl birleştirilir?

1

Basit / Monolitik

Hizmetlerin çoğu tek büyük çekirdekte

2

Katmanlı

Her katman yalnızca altındakini kullanır

3

Mikroçekirdek

Çekirdekte en az hizmet; gerisi kullanıcı alanında

4

Modüler

Çekirdek + çalışırken yüklenebilen modüller

5

Hibrit

Birden çok yaklaşımın birleşimi

Çekirdekte daha çok işlev → **Çekirdekte daha az işlev (mikroçekirdek yönü)**

Bu sıralama bir eksen değil, öğretim amaçlı bir gruplamadır. Kaynaklar: Silberschatz vd., 2.7; Tanenbaum & Bos, 1.7

Basit Yapı: MS-DOS Örneği

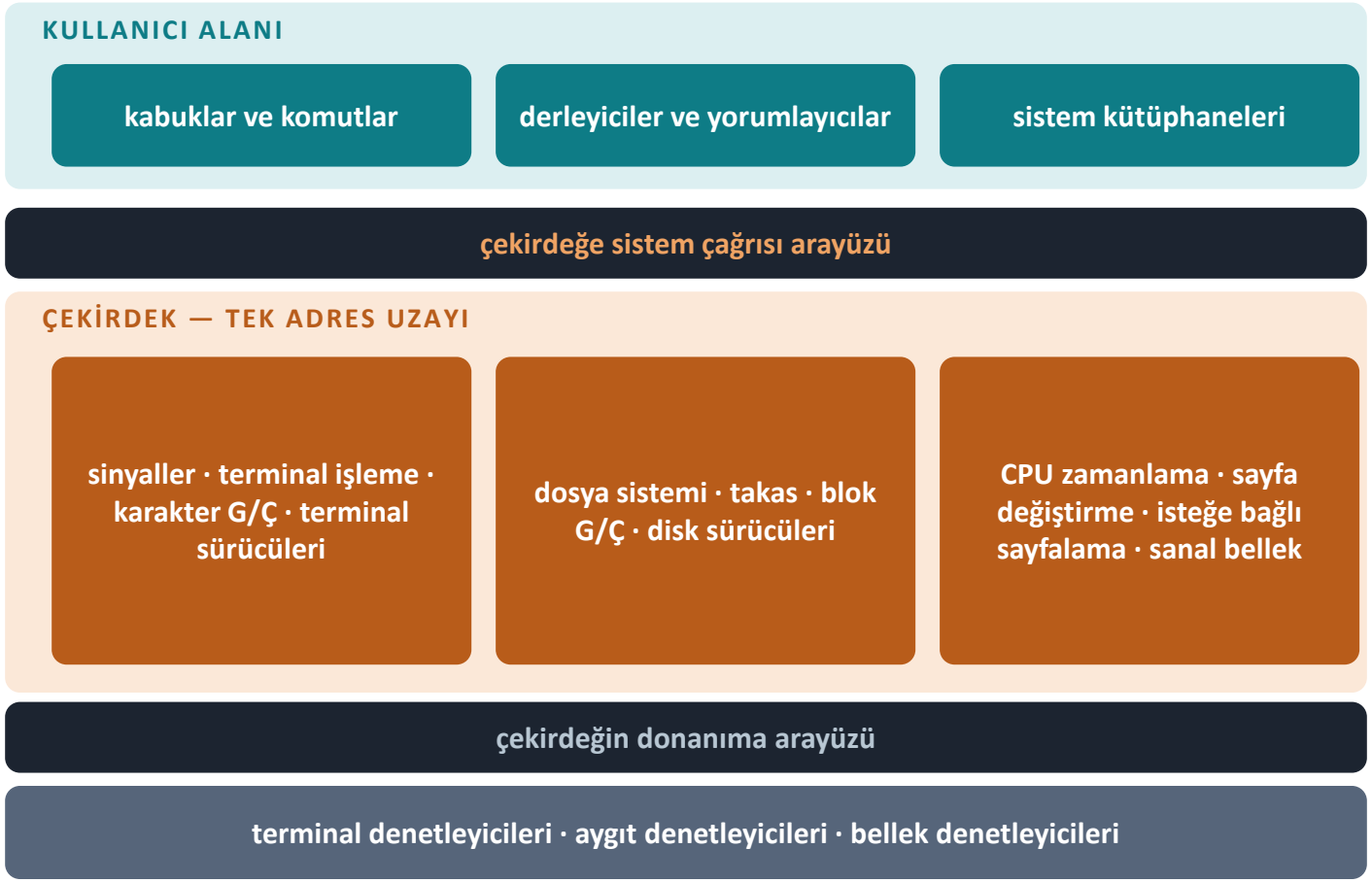


Uygulama alt katmanlara doğrudan erişebilir

- Az sayıda kişi tarafından, bu kadar yaygınlaşacağı düşünülmeden tasarlandı
- Amaç: en az alanda en çok işlev; modüllere dikkatle ayrılmadı
- Arayüzler ve işlev düzeyleri iyi ayrılmamıştır
- Hatalı ya da kötü niyetli bir program tüm sistemi çökertebilir
- **Neden? Intel 8088'de ikili mod ve donanım koruması yoktu (Hafta 1)**

Silberschatz vd., Bölüm 2.7.1 ve Şekil 2.11 temel alınmıştır

Monolitik Yapı: Geleneksel UNIX



Silberschatz vd., Şekil 2.12 sadeleştirilmiştir · Tanenbaum & Bos, Bölüm 1.7.1

Sistem çağrısı arayüzünün altı ile donanımın üstü arasındaki her şey çekirdektir.

Artı

Sistem çağrısı arayüzünde ve çekirdek içi iletişimde ek yük çok azdır → performans

Eksi

Çok fazla işlev tek düzeyde; gerçekleştirmesi ve bakımı zor. Çekirdekteki bir yordamdaki hata tüm sistemi çökertebilir.

Katmanlı Yaklaşım (Layered Approach)



Her katman: veri yapıları + üst katmanların çağırabildiği işlemler. Katman, alt katmanın işlemlerinin NASIL gerçekleştirildiğini bilmez; yalnızca NE yaptığını bilir.

Artı: kurma ve hata ayıklama kolaylığı

Katman 1 hata ayıklanır, sonra 2… Hata bulunursa, o katmandadır; alt katmanlar zaten doğrulanmıştır.

Eksi: katmanları tanımlamak zor, ek yük

Bir G/Ç çağrısı katmanlardan geçerken her katman ek yük ekler; çağrı daha uzun sürer.

Kaynak: Silberschatz vd., Bölüm 2.7.2 ve Şekil 2.13

Katmanlı Yapı: THE Sistemi ve Katman Sıralama Sorunu

Katman	İşlev
5	Operatör
4	Kullanıcı programları
3	G/Ç yönetimi
2	Operatör–süreç iletişimi
1	Bellek ve tambur yönetimi
0	İşlemci ayırma ve çoklu programlama

Dijkstra ve öğrencileri (1968), Eindhoven · Tanenbaum & Bos, Şekil 1-25

Hangi katman kimin altında olmalı?

Bellek yönetimi, yedek depolamayı (backing store) kullanır → yedek depolama sürücüsü bellek yönetiminin altında olmalı

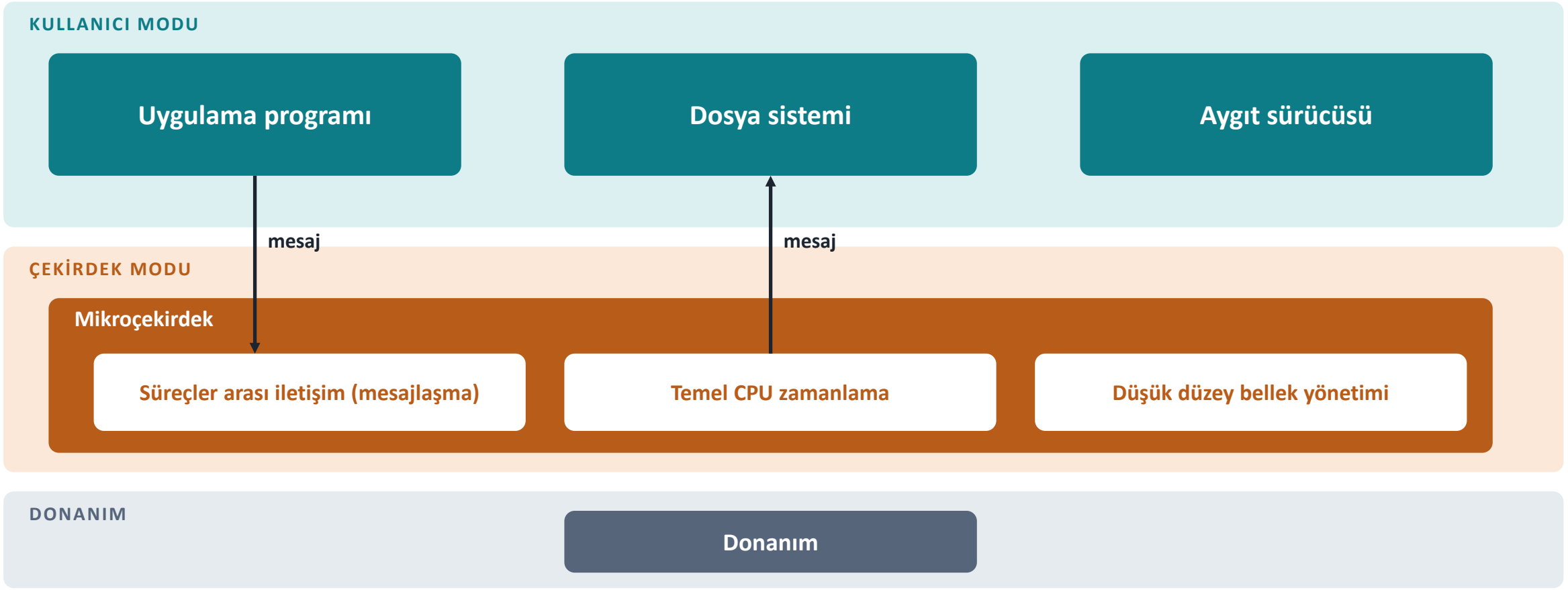
Sürücü G/Ç beklerken CPU başka sürece verilebilir → sürücü CPU zamanlayıcısının üstünde olmalı

Ama büyük sistemde CPU zamanlayıcısının bilgisi belleğe sığmayıp takas edilebilir → sürücü zamanlayıcının altında olmalı!

Sonuç: daha az ama daha işlevli katmanlarla tasarım eğilimi

Kaynaklar: Silberschatz vd., Bölüm 2.7.2; Tanenbaum & Bos, Bölüm 1.7.2

Mikroçekirdek (Microkernel) Mimarisi



Uygulama dosya okumak ister → mikroçekirdeğe mesaj → dosya sunucusu. İstemci ve hizmet doğrudan etkileşmez.

Silberschatz vd., Bölüm 2.7.3 ve Şekil 2.14 temel alınmıştır

Mikroçekirdek: Kazanımlar, Bedeller ve Örnekler

Olası kazanımlar

- Genişletilebilirlik: yeni hizmetler kullanıcı alanına eklenir, çekirdek değişmez
- Taşınabilirlik: küçük çekirdek başka donanıma daha kolay taşınır
- Güvenilirlik ve güvenlik: hizmetlerin çoğu kullanıcı süreci olarak çalıştığından bir hizmetin çökmesi sistemin geri kalanını genellikle etkilemez

Bedel: başarım

- Mesaj geçirme ve kullanıcı/çekirdek geçişleri ek yük getirir
- Örnek: Windows NT'nin ilk sürümü katmanlı mikroçekirdek düzenindeydi ve Windows 95'e göre yavaştı; NT 4.0'da katmanlar çekirdek alanına taşındı

ÖRNEK SİSTEMLER

Mach
Carnegie Mellon, 1980'lerin ortası

QNX
gömülü gerçek zamanlı İS

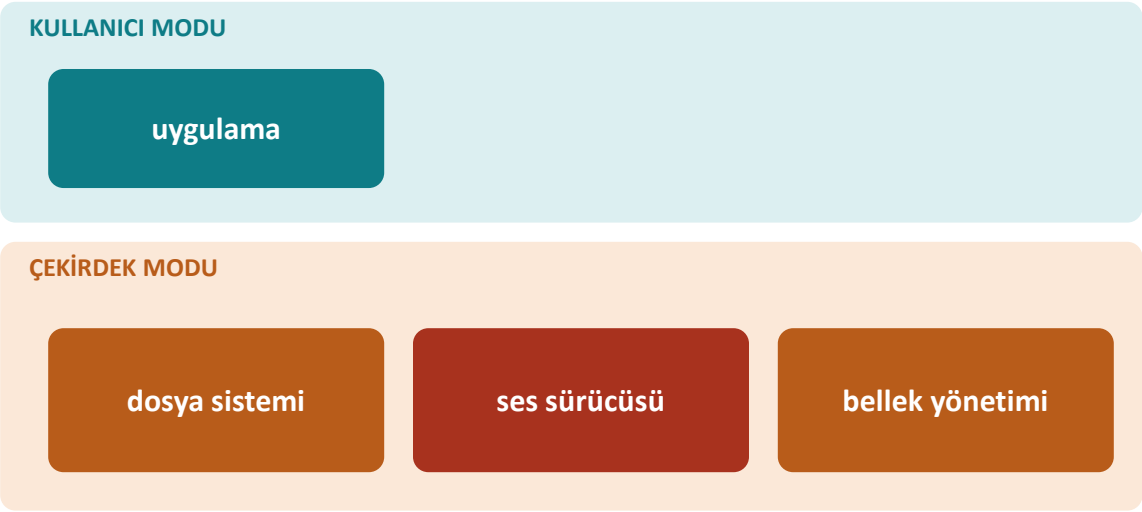
MINIX 3
sürücüler ayrı kullanıcı süreçleri

macOS çekirdeği
kısmen Mach tabanlı (Darwin)

Kazanımlar tasarıma ve gerçekleştirmeye bağlıdır; mutlak değildir. Kaynaklar: Silberschatz vd., 2.7.3; Tanenbaum & Bos, 1.7.3

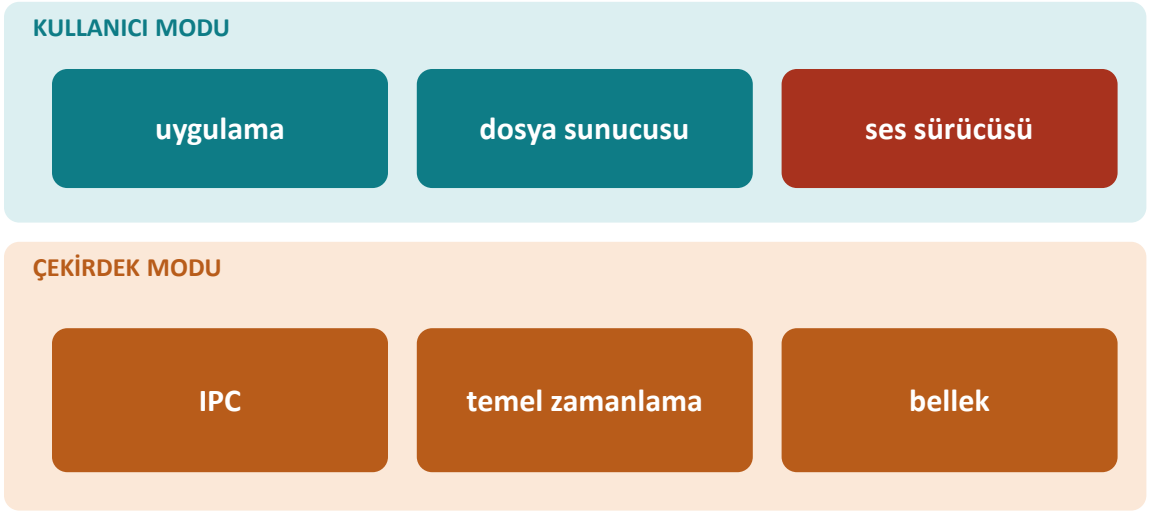
Senaryo: Ses Kartı Sürücüsü Çökerse Ne Olur?

Monolitik çekirdek



Sürücü çekirdekle aynı adres uzayında ve çekirdek modunda çalışır: geçersiz bir bellek erişimi tüm sistemi durdurabilir.

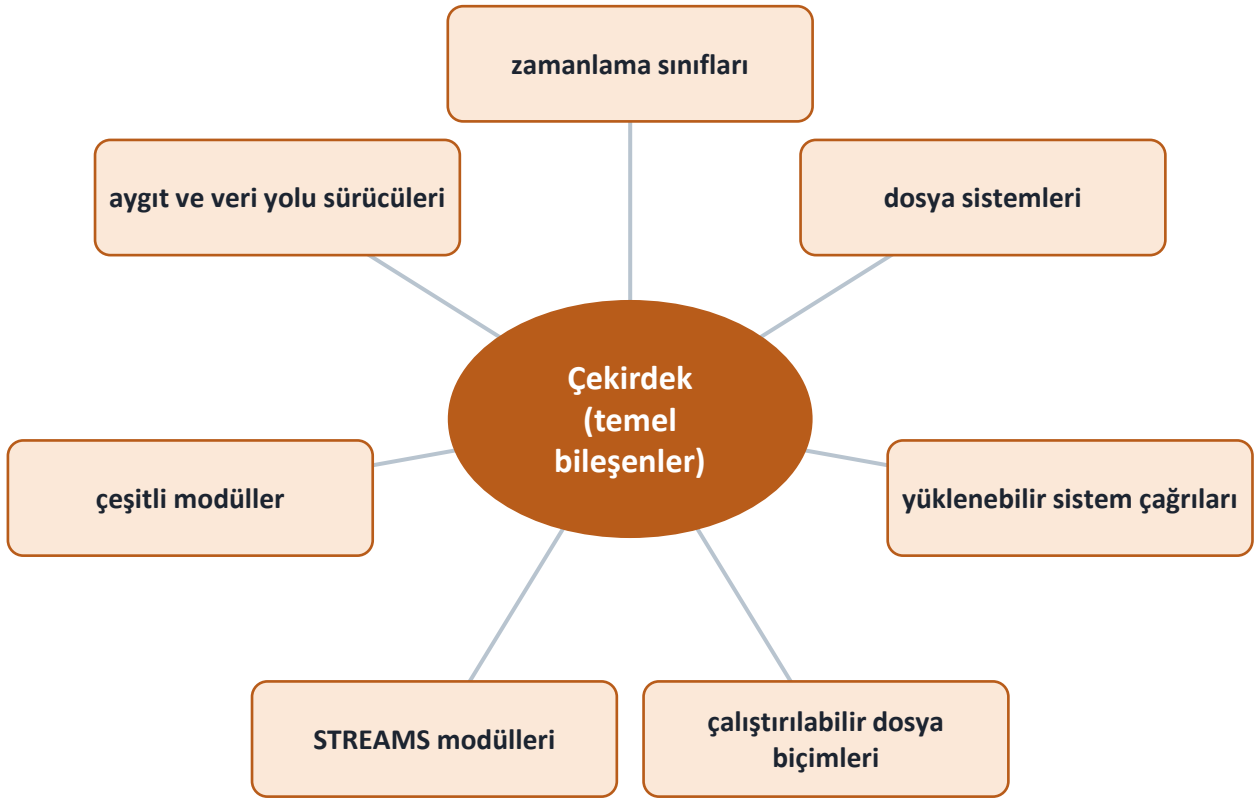
Mikroçekirdek



Sürücü ayrı bir kullanıcı sürecidir: ses bozulabilir ya da kesilebilir, ama sistemin geri kalanının çalışmaya devam etmesi beklenir.

Tanenbaum & Bos, Bölüm 1.7.3'teki ses sürücüsü örneği temel alınmıştır

Modüler Çekirdek: Yüklenebilir Çekirdek Modülleri

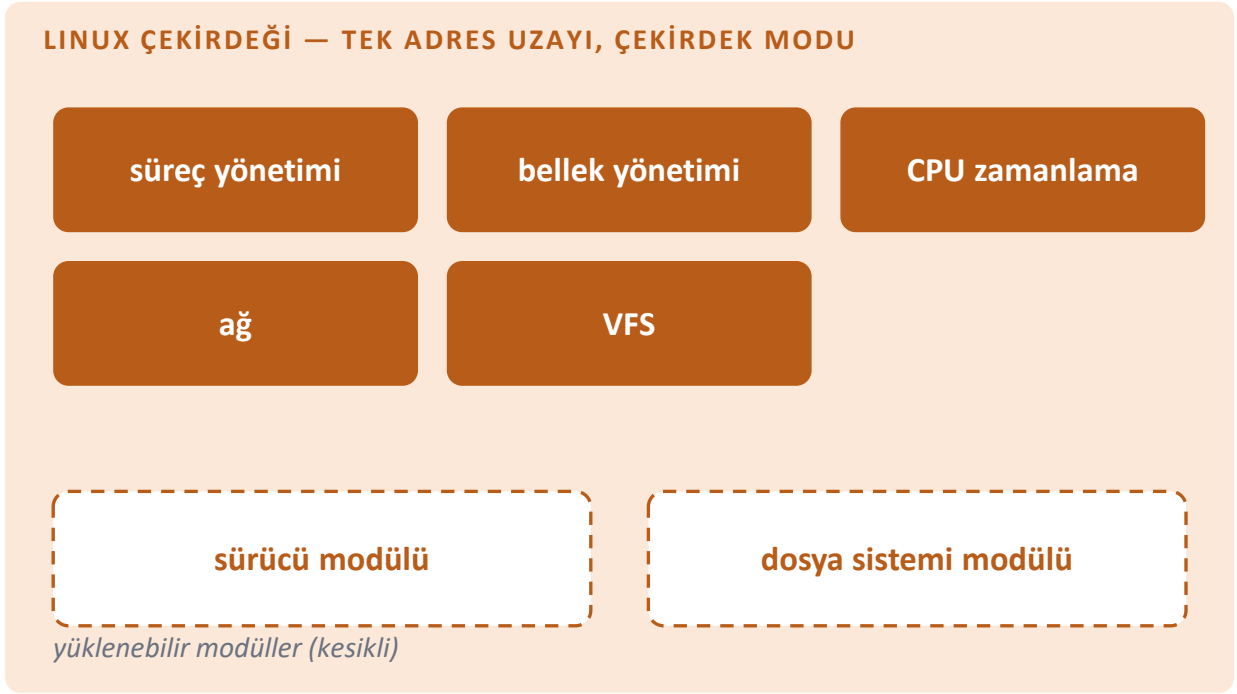


Solaris'in yedi modül türü

- Çekirdek temel bileşenleri içerir; ek hizmetler açılışta ya da çalışma sırasında modül olarak bağlanır
- Yeni özellik için çekirdeği yeniden derlemek gerekmez
- Katmanlıya benzer: her bölümün tanımlı, korunan arayüzü vardır — ama her modül diğerini çağırabilir
- Mikroçekirdeğe benzer: çekirdek küçük ve temeldir — ama modüller mesaj geçirmeden iletişir, daha verimlidir

Kaynak: Silberschatz vd., Bölüm 2.7.4 ve Şekil 2.15 · Örnekler: Solaris, Linux, macOS, Windows

Gerçek Sistem Örneği: Linux Monolitik mi, Modüler mi?



Monolitik yanı

İS'nin tek adres uzayında olması çok verimli başarım sağlar.

Modüler yanı

Yüklenebilir çekirdek modülleri (özellikle aygıt sürücüleri ve dosya sistemleri) çalışırken eklenip çıkarılabilir.

```
$ lsmod          # yüklü modülleri listele
$ sudo insmod modul.ko    # modül yükle
$ sudo rmmod modul        # modülü kaldır
```

Doğru ifade: Linux, modüler özellikler taşıyan monolitik bir çekirdektir.

Kaynak: Silberschatz vd., Bölüm 2.7.4–2.7.5 ve Bölüm 2 programlama projesi (Linux çekirdek modülleri)

Hibrit Sistemler: Saf Yapı Uygulamada Nadirdir

Pek az işletim sistemi tek ve katı biçimde tanımlanmış bir yapıyı benimser. Çoğu; başarım, güvenlik ve kullanılabilirlik için farklı yapıları birleştirir.

Linux, Solaris	**Monolitik**	**Modüler**		Tek adres uzayı başarım için; modüllerle dinamik işlev ekleme
Windows	**Büyük ölçüde monolitik**	**Modüler**	**Mikroçekirdeğe özgü davranışlar**	Kullanıcı modunda çalışan alt sistemler (İS "kişilikleri"); yüklenebilir çekirdek modülleri
macOS	**Katmanlı**	**Mach mikroçekirdeği + BSD**	**Çekirdek uzantıları**	Mach ve BSD bileşenleri aynı çekirdek ortamında birlikte bulunur

"Hibrit", öğretim amaçlı bir sınıflandırmadır; sistemler sürümden sürüme değişir. Kaynak: Silberschatz vd., Bölüm 2.7.5

Hibrit Örnekler: macOS ve iOS

macOS (kitapta: Mac OS X)

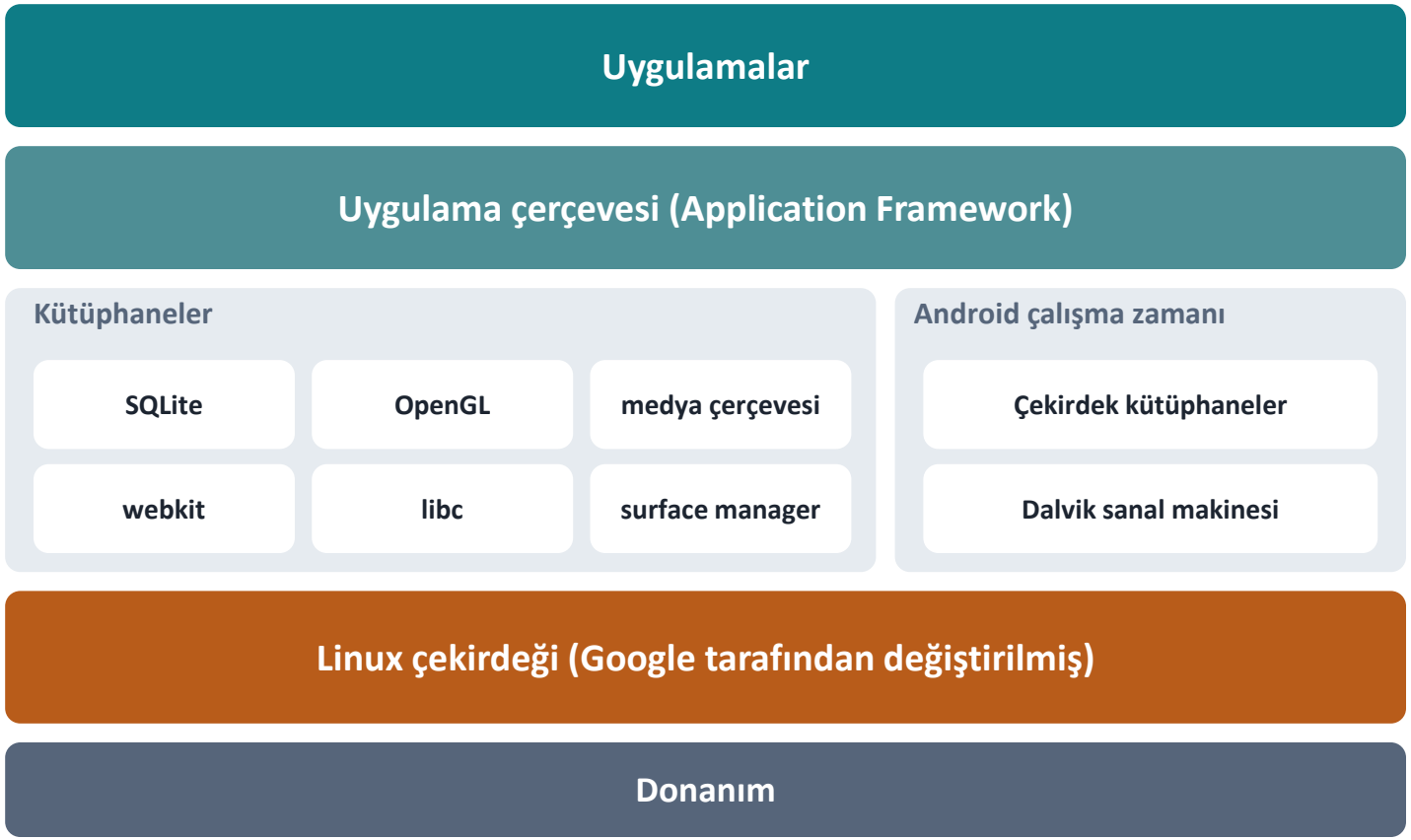


iOS



Kitabın (2013) gösterdiği yapıdır; güncel sürümlerde bileşen adları değişmiş olabilir. Silberschatz vd., Şekil 2.16–2.17

Hibrit Örnek: Android Yazılım Yığını



- Mobil uygulama geliştirmek için katmanlı bir yazılım yığını
- Linux çekirdeği: süreç, bellek ve aygıt sürücüsü desteği; güç yönetimi eklenmiş
- Uygulamalar Java diliyle, Google'ın ayrı Android API'si kullanılarak geliştirilir
- Açık kaynaklıdır ve farklı mobil donanımlarda çalışır

Kitabın (2013) gösterdiği yapıdır; güncel Android sürümlerinde çalışma zamanı bileşenleri değişmiştir. Silberschatz vd., Şekil 2.18

Yapıların Karşılaştırması

Yapı	Başarım	Modülerlik	Hata yalıtımı	Genişletilebilirlik	İletişim ek yükü	Örnek
Monolitik	Genellikle yüksek	Düşük	Zayıf: çekirdek hatası sistemi etkileyebilir	Çekirdek değişikliği gerekir	Düşük	Geleneksel UNIX
Katmanlı	Katman geçişleri yük ekler	Yüksek	Orta	Katman tanımları sınırlar	Katman başına artar	THE
Mikroçekirdek	Mesajlaşma nedeniyle düşebilir	Yüksek	Genellikle daha iyi	Yüksek: hizmetler kullanıcı alanında	Yüksek	Mach, QNX, MINIX 3
Modüler	Yüksek	Yüksek	Modüller çekirdek modunda çalışır	Yüksek: çalışırken modül yükleme	Düşük	Solaris, Linux
Hibrit	Tasarıma bağlı	Tasarıma bağlı	Tasarıma bağlı	Genellikle yüksek	Tasarıma bağlı	Windows, macOS, Android

Tablo genel eğilimleri gösterir; gerçek sonuç gerçekleştirmeye bağlıdır. Örneğin bir mikroçekirdek “her zaman daha güvenli” ya da monolitik çekirdek “her zaman daha hızlı” değildir.

Sistem Önyüklemesi (System Boot): Adım Adım



İS açısından mantık: RAM açılışta bilinmeyen durumdadır; bu yüzden ilk kod kalıcı bellekte (ROM) bulunur. Her aşama, bir sonraki daha büyük programı bulup yükleyecek kadar bilgiye sahiptir.

Kaynak: Silberschatz vd., Bölüm 2.10

İşletim Sistemi Nerede Saklanır?

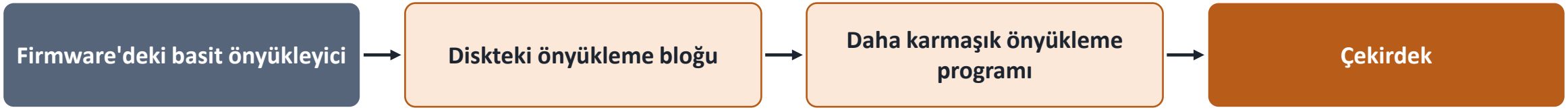
İS tamamen donanım yazılımında (ROM/EPROM)

- Küçük İS'ler ve basit donanım için uygundur: ör. bazı telefonlar, oyun konsolları
- Sorun: ROM'daki kodu değiştirmek zordur; donanım yazılımında kod çalıştırmak RAM'e göre yavaştır ve bu bellek pahalıdır

Önyükleyici firmware'de, İS diskte

- Windows, macOS, UNIX gibi büyük genel amaçlı sistemler
- Firmware yalnızca diskteki önyükleme bloğunu okuyup çalıştırır; disk üzerindeki önyükleme kodu ve İS kolayca güncellenebilir

İKİ AŞAMALI ÖNYÜKLEME (ör. PC'ler)



Güncel Sistem Örneği: PC'lerde tarihsel donanım yazılımı BIOS'tu; yeni bilgisayarların çoğu UEFI kullanır (Tanenbaum & Bos, 11.3.2).

Kaynak: Silberschatz vd., Bölüm 2.10

Tamamlayıcı Konular: İS Hata Ayıklama ve İS Üretimi

İşletim sistemi hata ayıklama

- Süreç hatası: hata günlüğe yazılır; bellek görüntüsü (core dump) dosyaya alınabilir
- Çekirdek hatası (crash): bellek durumu crash dump olarak saklanır
- Başarım ayarlama: darboğazları bulmak için izleme araçları (ör. top, DTrace)

Kernighan Yasası: “Hata ayıklamak, kodu yazmaktan iki kat zordur.”

İS üretimi (SYSGEN)

İS, belirli bir bilgisayar için yapılandırılır. Belirlenmesi gerekenler:

- CPU ve seçenekleri
- Kullanılabilir bellek miktarı
- Aygıtlar ve kesme numaraları
- İS seçenekleri (ör. tampon sayısı, CPU zamanlama algoritması)

Kaynak koddan tam derleme | **Hazır modüllerden seçip bağlama** | **Tamamen tablo güdümlü**

en özel ↔ en genel

Kaynak: Silberschatz vd., Bölüm 2.8–2.9 (kısa tamamlayıcı içerik)

Bu Hafta Neler Öğrendik?

1. İS, kullanıcıya ve sisteme yönelik hizmetler sunar
2. Kullanıcı bu hizmetlere kabuk ya da GUI ile ulaşır; kabuk çekirdek değildir
3. Programlar hizmeti sistem çağrısıyla ister: tuzak → çekirdek modu → dönüş
4. API programcının arayüzüdür; her API çağrısı doğrudan sistem çağrısı değildir
5. Politika “ne”, mekanizma “nasıl” sorusunu yanıtlar
6. Yapılar: monolitik, katmanlı, mikroçekirdek, modüler; gerçek sistemler çoğunlukla hibrittir

Gelecek hafta: fork() ve exec() ile ne oluşur? → Süreç kavramı, süreç işlemleri ve süreçler arası iletişim

Yaygın Yanlış Anlamalar

YANLIŞ	DOĞRU
“Kabuk = çekirdek”	Kabuk, kullanıcı modunda çalışan bir komut yorumlayıcısı programıdır; çekirdek ise İS'nin çekirdek modundaki kısmıdır
“API = sistem çağrısı”	API programcının arayüzüdür; sistem çağrısı çekirdeğe geçiş mekanizmasıdır. Bir API işlevi hiç sistem çağrısı yapmayabilir
“printf() doğrudan çekirdeğe girer”	printf() kütüphane işlevidir; asıl G/Ç için write() sistem çağrısını kullanır
“Sistem programı = sistem çağrısı”	cp gibi sistem programları kullanıcı modunda çalışır ve işini sistem çağrılarıyla yapar
“Linux saf, basit bir monolitik çekirdektir”	Linux monolitiktir ama yüklenebilir modüllerle modüler özellikler taşır
“Mikroçekirdek her zaman daha güvenli ve iyidir”	Hata yalıtımı genellikle daha iyidir; ama mesajlaşma ek yükü getirir ve sonuç gerçekleştirmeye bağlıdır

Kontrol Soruları

KAVRAMSAL

S1 **Bir program printf() çağırdığında doğrudan çekirdeğe mi girer? API, kütüphane ve sistem çağrısı kavramlarıyla açıklayınız.**

İlgili bölüm: 3 · Sistem çağrıları

S2 **Politika ile mekanizma ayrımı neden işletim sistemi tasarımını kolaylaştırır? Bir örnek veriniz.**

İlgili bölüm: 6 · Tasarım ve gerçekleştirme

DEĞERLENDİRME

Kontrol Soruları

KARŞILAŞTIRMA

S3 **Sistem çağrısı ile sistem programı arasındaki farkı; nerede çalıştıkları ve kimin kullandığı açısından karşılaştırınız.**

İlgili bölüm: 5 · Sistem programları

S4 **Katmanlı yaklaşım ile modüler çekirdek yaklaşımını karşılaştırınız. Modüler yapı katmanlıdan hangi açıdan daha esnektir?**

İlgili bölüm: 7 · İS yapıları

Kontrol Soruları

AKIŞ / SIRALAMA

S5 **Bir read() çağrısında şu adımları doğru sıraya koyunuz: tuzak · sistem çağrısı numarasının yazmaca konması · kullanıcı moduna dönüş · tablo ile işleyici seçimi · parametrelerin hazırlanması**

İpucu: Hangi adımlar kullanıcı modunda, hangileri çekirdek modunda gerçekleşir?

S6 **Güç düğmesine basıldıktan kullanıcı ortamı açılana kadar geçen önyükleme aşamalarını sıralayınız.**

İlgili bölüm: 8 · Önyükleme

Kontrol Soruları

KISA SENARYO

S7 **Mikroçekirdek yaklaşımında bir aygıt sürücüsü çökerse, monolitik çekirdeğe göre olası etkisi neden farklı olabilir?**

İpucu: Sürücü hangi modda ve hangi adres uzayında çalışıyor?

S8 **Bir sistem çağrısı için 6 parametre gerekiyor, ama işlemcide bu amaçla yalnızca 4 yazmaç ayrılmış. İS tasarımcısı olarak hangi yöntemi seçersiniz? Neden?**

İlgili bölüm: 3 · Parametre aktarma

Kaynaklar

ANA KAYNAK

Silberschatz, A., Galvin, P. B., & Gagne, G. (2013). *Operating System Concepts* (9th ed.). John Wiley & Sons.

Bu hafta: Bölüm 2 — 2.1–2.7, 2.10; 2.8–2.9 kısaca · Hatırlatma: 1.5

DESTEKLEYİCİ KAYNAK

Tanenbaum, A. S., & Bos, H. (2015). *Modern Operating Systems* (4th ed.). Pearson.

Bu hafta: Bölüm 1 — 1.5.6, 1.6 (1.6.1–1.6.5), 1.7 · Önyükleme notu: 11.3.2

Not: Bu sunumdaki diyagramlar, belirtilen yerlerde kaynak kitaplardaki şekiller temel alınarak yeniden çizilmiştir.